CMPUT 267: Machine Learning I · Fall 2026

Midterm 2 Review

Xi Ye

November 3, 2026

Recall: ERM

We aim to minimize the **(unknown) expected risk**:

$$L(f) = \mathbb{E}_{\mathbf{X},Y}[\ell(f(\mathbf{X}), Y)].$$

In practice, we estimate it using the **empirical risk** on the dataset D and minimize that estimate:

$$\widehat{L}_{\mathcal{D}}(f) = \frac{1}{n}\sum_{i=1}^{n} \ell(f(\mathbf{x}_i), y_i), \qquad \widehat{f} \in \arg\min_{f \in \mathcal{F}} \widehat{L}_{\mathcal{D}}(f).$$

Recall: Convex Optimization

A convex objective is **bowl shaped**. Every local minimum is a global minimum.

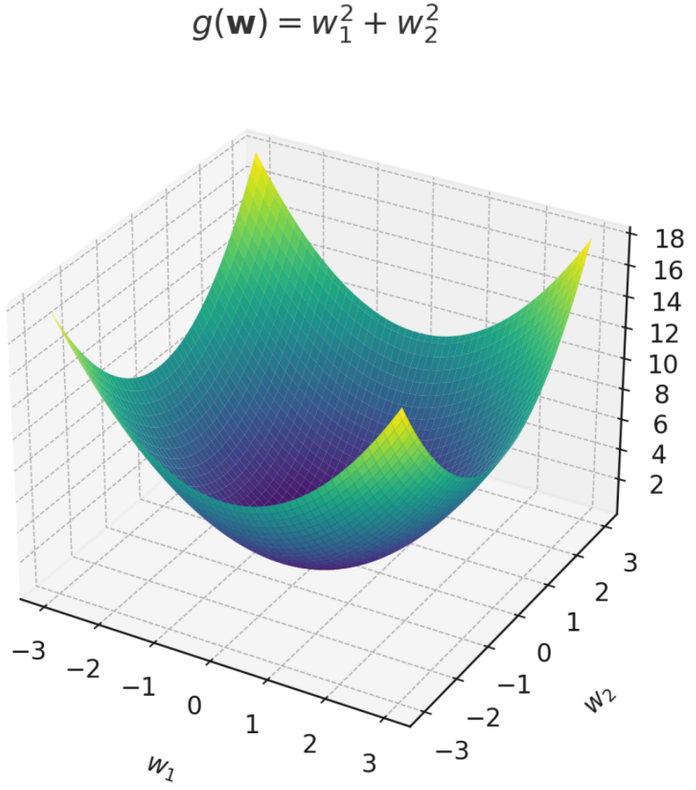
On a convex domain, for a twice-differentiable objective:

One parameter

$$g''(w) \geq 0 \quad \text{everywhere.}$$

Several parameters

The **Hessian**, the matrix of second partial derivatives, must be **positive semidefinite** everywhere.



Minimizing a Convex Function

For a **differentiable convex objective**, every stationary point is a global minimizer.

One parameter on $\mathbb{R}$: set the derivative to zero.

$$g'(w^*) = 0.$$

Several parameters on $\mathbb{R}^d$: solve all partial-derivative equations together.

$$\nabla g(\mathbf{w}^*) = \mathbf{0} \quad \Longleftrightarrow \quad \frac{\partial g}{\partial w_j}(\mathbf{w}^*) = 0 \quad \text{for every } j.$$

Practice Q20: A convex function in two variables

Consider the convex function

$$h(w_1, w_2) = 2w_1^2 + 2w_1w_2 + 3w_2^2 - 10w_2 + 12, \qquad (w_1, w_2) \in \mathbb{R}^2.$$

A. The unique minimizer is $(w_1, w_2) = (-1, 2)$.

B. The minimum value of h is 2.

C. The unique minimizer is $(w_1, w_2) = (1, -2)$.

D. The minimum value of h is 12.

Linear Regression: Closed-Form Solution

For linear regression with **squared loss**, compute:

$$X \leftarrow \begin{bmatrix} \mathbf{x}_1^\top \\ \vdots \\ \mathbf{x}_n^\top \end{bmatrix}, \qquad \mathbf{y} \leftarrow \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix},$$

$$A \leftarrow X^\top X, \qquad \mathbf{b} \leftarrow X^\top \mathbf{y},$$

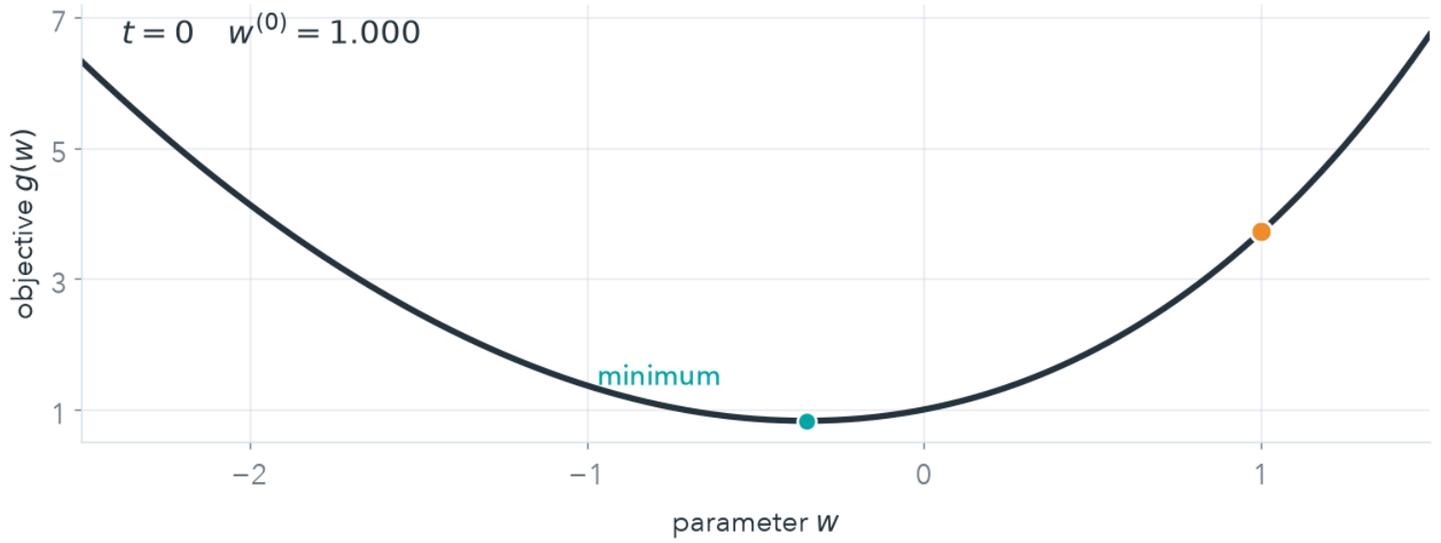
$$\widehat{\mathbf{w}} \leftarrow A^{-1}\mathbf{b} = (X^\top X)^{-1} X^\top \mathbf{y},$$

$$\widehat{f}(\mathbf{x}) \leftarrow \mathbf{x}^\top \widehat{\mathbf{w}}.$$

Setting the gradient to zero gives the **normal equations** $A\widehat{\mathbf{w}} = \mathbf{b}$. The inverse formula assumes $X^\top X$ is invertible.

Gradient Descent: Iterative Search

Gradient descent finds an approximate minimizer by **iterative search**, even when a closed-form solution is unavailable.



Start from $w^{(0)} = 1$ and repeatedly use the local slope to move downhill.

First-Order and Second-Order Updates

Gradient descent

$$w^{(t+1)} = w^{(t)} - \eta^{(t)} g'(w^{(t)})$$

Uses the **first derivative** and a chosen learning rate.

Newton's method

$$w^{(t+1)} = w^{(t)} - \frac{g'(w^{(t)})}{g''(w^{(t)})}$$

Uses the **first and second derivatives**. The second derivative determines the step scale.

Newton's step requires $g''(w^{(t)}) \neq 0$. Neither method guarantees a lower objective after an arbitrary step.

Multivariate Gradient Descent

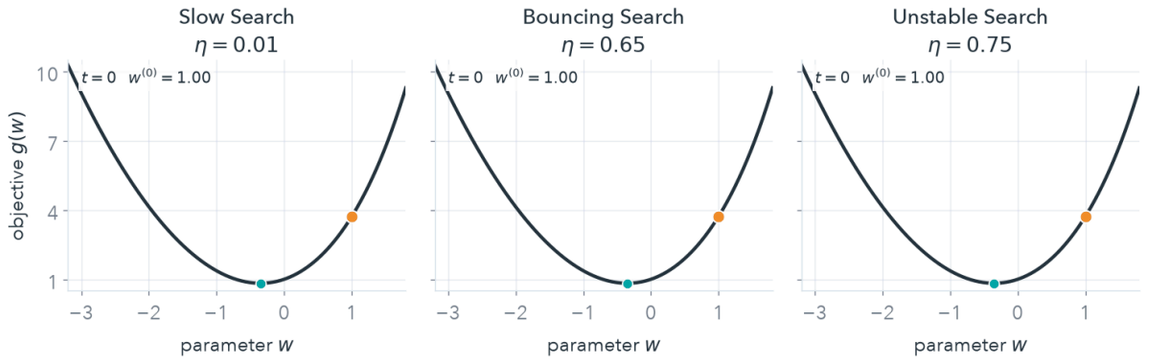
The scalar update extends by replacing the derivative with the gradient:

$$\boxed{\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta^{(t)} \nabla g(\mathbf{w}^{(t)})}$$

Each coordinate updates simultaneously:

$$w_j^{(t+1)} = w_j^{(t)} - \eta^{(t)} \frac{\partial g}{\partial w_j}(\mathbf{w}^{(t)}).$$

Learning-Rate Tradeoffs



Smaller learning rate

- Shorter, safer updates
- More iterations to reach a useful solution
- Less likely to jump across a narrow minimum

Larger learning rate

- Faster movement when the scale is appropriate
- Can cross the minimum repeatedly
- Can increase the objective and diverge

Practice Q2: A simultaneous gradient update

We want to minimize

$$J(w_0, w_1) = \tfrac{1}{2}[(w_0 - 1)^2 + (w_0 + 2w_1 - 3)^2].$$

Start at $(w_0, w_1) = (0, 0)$ and take one simultaneous gradient-descent update with step size $\eta = 1/2$.

A. The gradient at the starting point is $(-4, -6)^\top$.

B. The new parameters are $(2, 3)$.

C. This update reduces J from 5 to a value below 5.

D. After updating w_0, recomputing the derivative for w_1 using that new w_0 implements the same simultaneous update.

Mini-Batch Gradient Descent

Input: dataset $\mathcal{D}$, learning-rate schedule, T **epochs** (passes over the dataset), batch size b

1. Initialize $\mathbf{w}$.
2. For each pass $t = 1, \ldots, T$:
   - **shuffle the training examples**.
   - form $M = \mathrm{floor}(n/b)$ full mini-batches and discard the remainder.
   - for $m = 1, \ldots, M$:
     - compute $\mathbf{g} \leftarrow \nabla \widehat{L}_{B_m}(\mathbf{w})$.
     - update $\mathbf{w} \leftarrow \mathbf{w} - \eta \mathbf{g}$.
3. Return $\widehat{f}(\mathbf{x}) = f_{\mathbf{w}}(\mathbf{x})$.

Note: This algorithm does not handle the remainder. The final incomplete batch is discarded.

Cost per Update and per Epoch

For d parameters:

Method	Batch size	Updates per epoch	Cost per update	Cost per epoch
Batch GD	n	1	$O(nd)$	$O(nd)$
Mini-batch GD	b	$\text{floor}(n/b)$	$O(bd)$	$O(nd)$
Stochastic GD	1	n	$O(d)$	$O(nd)$

All three process about n examples per epoch. The cost per epoch is the same. But smaller batch size induces more frequent updates per epoch.

Practice Q13: Reviewing mini-batch gradient descent

A student writes the following code for one epoch of mini-batch gradient descent with squared loss and $f_w(x) = x^\top w$. The dataset D has n examples, b divides n, and $\eta > 0$. Each batch contains b examples; `g = 0` sets the gradient vector to zero.

```
1  shuffle(D)
2  for each batch B of size b:
3      g = 0
4      for each (x, y) in B:
5          g = g + 2*(dot(x,w)-y)*x
6          w = w - (eta/b)*g
```

A. The code correctly implements mini-batch gradient descent for every batch size b.

B. Moving line 6 outside the loop on line 4, but keeping it inside the loop on line 2, gives the correct mini-batch update.

C. As written, the code updates w exactly n/b times in one epoch.

D. When $b = 1$, the code gives the usual stochastic gradient-descent update for each example.

Polynomial Regression

Polynomial regression allows **higher-degree terms** such as x_1^3 and **interactions** such as $x_1 x_2$.

The degree-p feature map collects all products with total degree at most p:

$$\phi_p(\mathbf{x}) = \left(x_1^{a_1} \cdots x_d^{a_d}\right)_{\substack{a_j \in \{0,1,2,\ldots\}, \\ a_1 + \cdots + a_d \le p}}, \qquad f_{\mathbf{w}}(\mathbf{x}) = \mathbf{w}^\top \phi_p(\mathbf{x}).$$

Example: two inputs x_1, x_2, degree 3.

$$\phi_3(x_1, x_2) = (1, x_1, x_2, x_1^2, x_1 x_2, x_2^2, x_1^3, x_1^2 x_2, x_1 x_2^2, x_2^3)^\top,$$
$$f_{\mathbf{w}}(x_1, x_2) = \mathbf{w}^\top \phi_3(x_1, x_2), \qquad \mathbf{w} \in \mathbb{R}^{10}.$$

The predictor can be nonlinear in the inputs, but remains **linear in the weights**.

Nested Classes and Training Loss

Allowing a higher degree gives a larger set of candidate predictors.

$$\mathcal{F}_1 \subseteq \mathcal{F}_2 \subseteq \mathcal{F}_3 \subseteq \cdots$$

$$\min_{f \in \mathcal{F}_1} \widehat{L}_{\mathcal{D}}(f) \geq \min_{f \in \mathcal{F}_2} \widehat{L}_{\mathcal{D}}(f) \geq \min_{f \in \mathcal{F}_3} \widehat{L}_{\mathcal{D}}(f) \geq \cdots$$



More flexibility allows a better fit to the training data.

Practice Q14: A restricted feature map

A learner uses $\phi(x) = (1, x, x^3)^\top$ and $f_w(x) = \phi(x)^\top w$, where $x \in \mathbb{R}$ and all three weights are unrestricted real numbers. Let $\mathcal{H}$ be this function class and $\mathcal{F}_3$ the class of all polynomials of degree at most 3.

A. Every polynomial of degree 2 belongs to $\mathcal{H}$.

B. The predictor is linear in the weights, even though it need not be linear in x.

C. The function $2 - x + 3x^3$ belongs to $\mathcal{H}$.

D. On every dataset, $\min_{f \in \mathcal{H}} \hat{L}(f) \leq \min_{f \in \mathcal{F}_3} \hat{L}(f)$.

Chapter 7

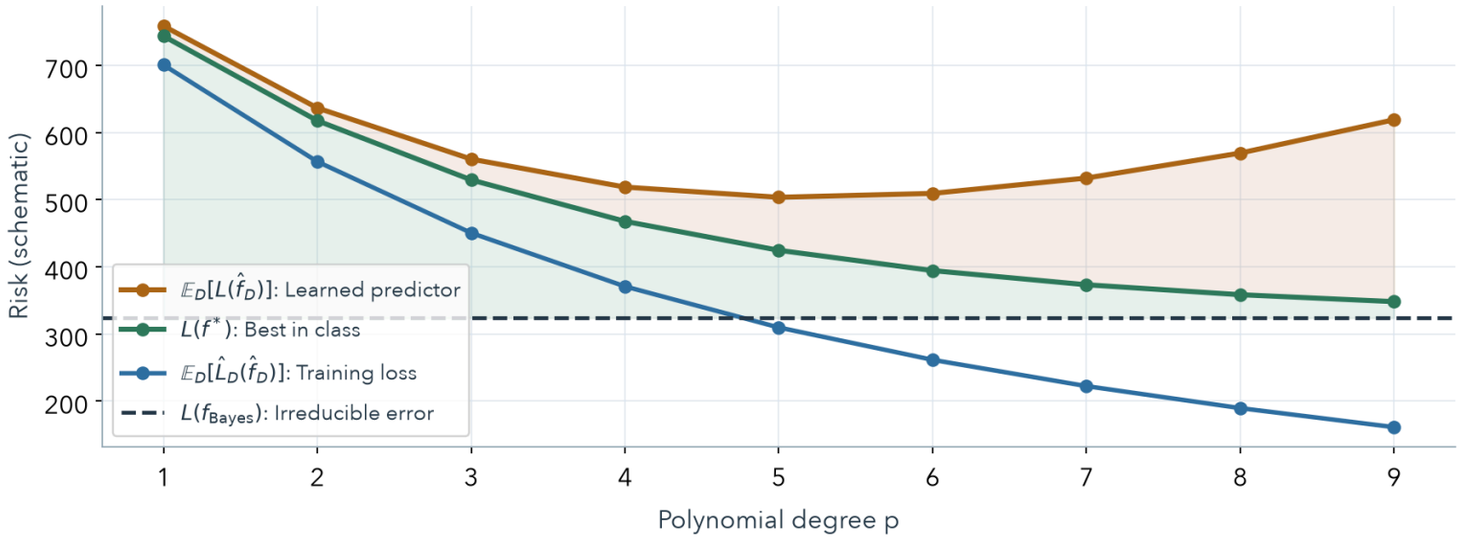
Decomposing Expected Risk

$$\mathbb{E}_D[L(\widehat{f}_D)] = \underbrace{\mathbb{E}_D[L(\widehat{f}_D)] - L(f^*)}_{\text{Estimation error}} + \underbrace{L(f^*) - L(f_{\text{Bayes}})}_{\text{Approximation error}} + \underbrace{L(f_{\text{Bayes}})}_{\text{Irreducible error}} .$$

- f_{Bayes}: the predictor with the smallest expected risk among **all functions** of the available inputs.
- f^*: the predictor with the smallest expected risk **within our chosen function class** $\mathcal{F}$.

Error and Polynomial Degree

Fixed training-data size; increasing polynomial degree (schematic).

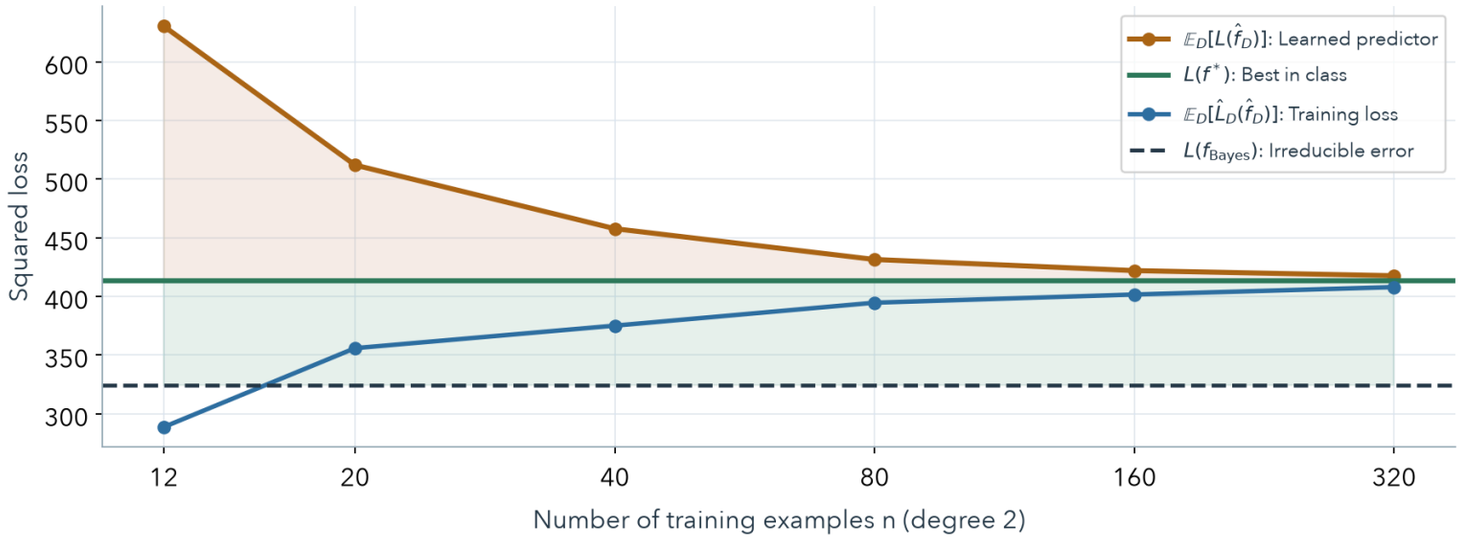


$$\underbrace{\text{EE} = \mathbb{E}_D[L(\widehat{f}_D)] - L(f^*)}_{\text{Estimation: orange minus green}} \qquad \underbrace{\text{AE} = L(f^*) - L(f_{\text{Bayes}})}_{\text{Approximation: green minus dashed}} .$$

A richer class shrinks the approximation gap, but can widen the estimation gap.

Error and Training-Data Size

Keep **degree** fixed and increase the number of training examples.



$$\underbrace{\mathrm{EE} = \mathbb{E}_D[L(\widehat{f}_D)] - L(f^*)}_{\text{Estimation: orange minus green}} \qquad \underbrace{\mathrm{AE} = L(f^*) - L(f_{\mathrm{Bayes}})}_{\text{Approximation: green minus dashed}} .$$

More training data shrinks the estimation gap; the approximation gap stays fixed.

Summary: Class Size and Data Size

Error component	Higher polynomial degree	More training examples
Irreducible error	Unchanged	Unchanged
Approximation error	Cannot increase	Unchanged
Estimation error	Often increases	Usually decreases

Training, Validation, and Test Sets

Reserve a third portion, the **test set**, to evaluate the final choice.

All available data

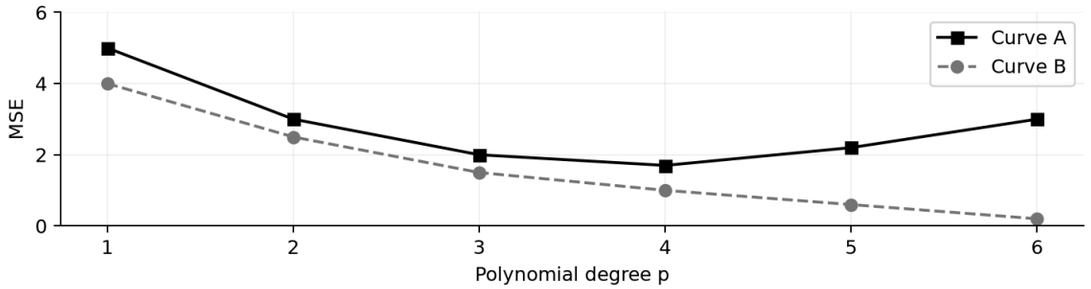
Training	Validation	Test
Fit coefficients	Choose degree or stopping point	Evaluate final predictor



Final evaluation uses data involved in **neither fitting parameters nor choosing models**.

Practice Q19: Identifying two error curves

One curve is training MSE and the other is validation MSE. Each predictor is an exact unregularized least-squares fit on the same training set, using all polynomials of degree at most p. The validation set is fixed and disjoint from training. The curves have lost their labels.



A. Curve A must be the training MSE.

B. Curve B must be the training MSE.

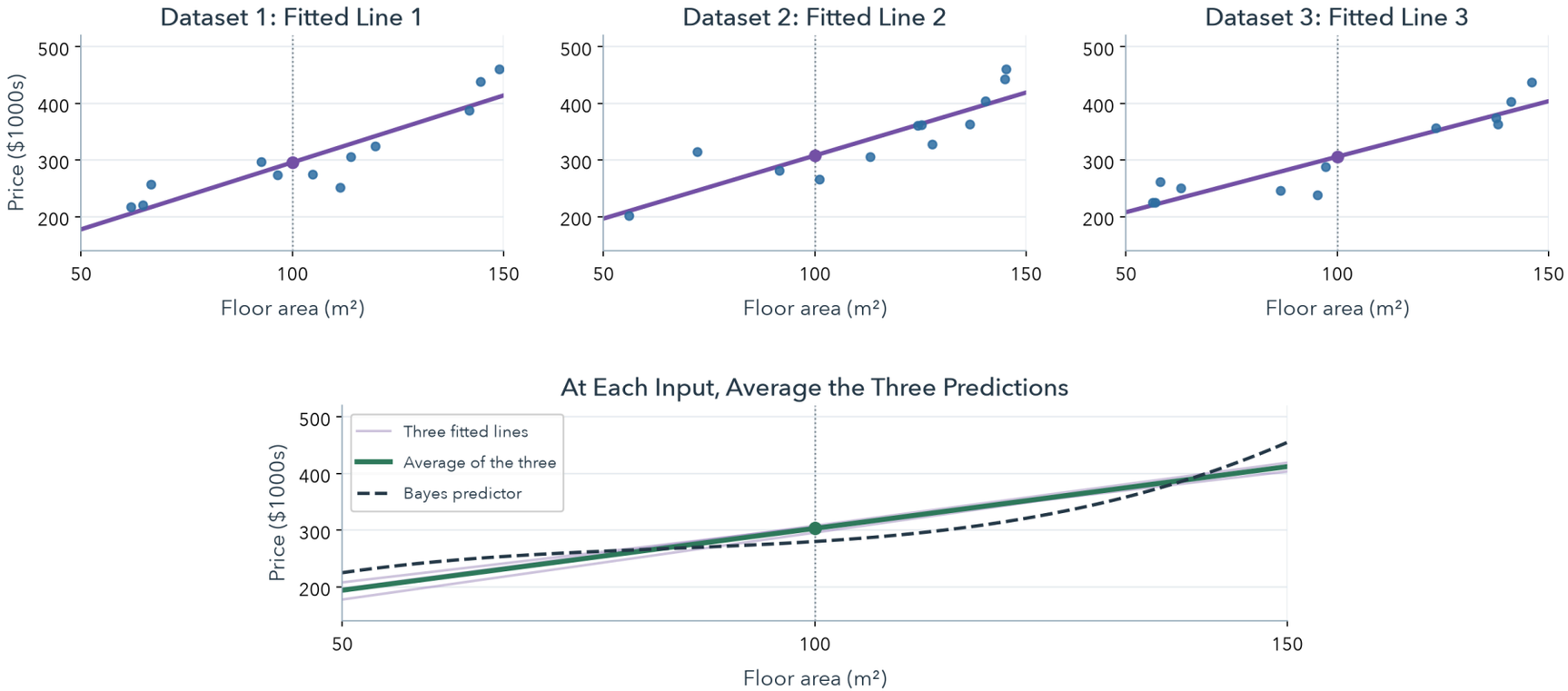
C. Using the validation curve to select among the plotted degrees gives $p = 4$.

D. The rise in Curve A from $p = 4$ to $p = 6$ proves that the optimizer failed to minimize training MSE.

Recall: The Average Predictor

Train the **same learner** on different datasets of the **same size**.

$$\bar{f}(\mathbf{x}) = \mathbb{E}_D[\hat{f}_D(\mathbf{x})].$$



Bias, Variance, and Irreducible Error

Another decomposition: **bias, variance, and irreducible error**.

(This requires squared loss: $\ell(\hat{y}, y) = (\hat{y} - y)^2$.)

$$\mathbb{E}_D[L(\widehat{f}_D)] = \underbrace{\mathbb{E}_{\mathbf{X}}[(\bar{f}(\mathbf{X}) - f_{\text{Bayes}}(\mathbf{X}))^2]}_{\text{Bias}}$$

$$+ \underbrace{\mathbb{E}_{\mathbf{X}}[\text{Var}_D(\widehat{f}_D(\mathbf{X}))]}_{\text{Variance}} + \underbrace{L(f_{\text{Bayes}})}_{\text{Irreducible error}} .$$

Here, $\bar{f}$ averages predictions across training datasets.

Bias: average prediction versus Bayes prediction.

Variance: fitted predictions versus their average.

Irreducible error: label randomness that remains at a fixed input.

Penalizing the Weights

For squared loss, penalize weights $w_1, \ldots, w_d$, leaving the intercept w_0 unpenalized.

$$\widehat{L}_\lambda(\mathbf{w}) = \underbrace{\frac{1}{n}\sum_{i=1}^{n}(\mathbf{x}_i^\top \mathbf{w} - y_i)^2}_{\text{Training MSE}} + \underbrace{\frac{\lambda}{n}\sum_{j=1}^{d} w_j^2}_{\text{Weight penalty}} .$$

The nonnegative hyperparameter λ controls penalty strength. This squared-loss, squared-weight method is **ridge regression**.

Chapter 8

Recall: Maximum Likelihood Estimation

MLE chooses θ to maximize the likelihood of the observed dataset $\mathcal{D}$.

For i.i.d. observations $\mathcal{D} = (y_1, \ldots, y_n)$:

$$\begin{aligned}
\hat{\theta}_{\text{MLE}} &= \arg\max_{\theta} p(\mathcal{D} \mid \theta) \\
&= \arg\max_{\theta} \prod_{i=1}^{n} p_Y(y_i \mid \theta) \\
&= \arg\min_{\theta} \underbrace{\left[-\sum_{i=1}^{n} \log p_Y(y_i \mid \theta) \right]}_{\text{Negative log likelihood}}.
\end{aligned}$$

Logs turn products into sums. Negation changes maximization to minimization.

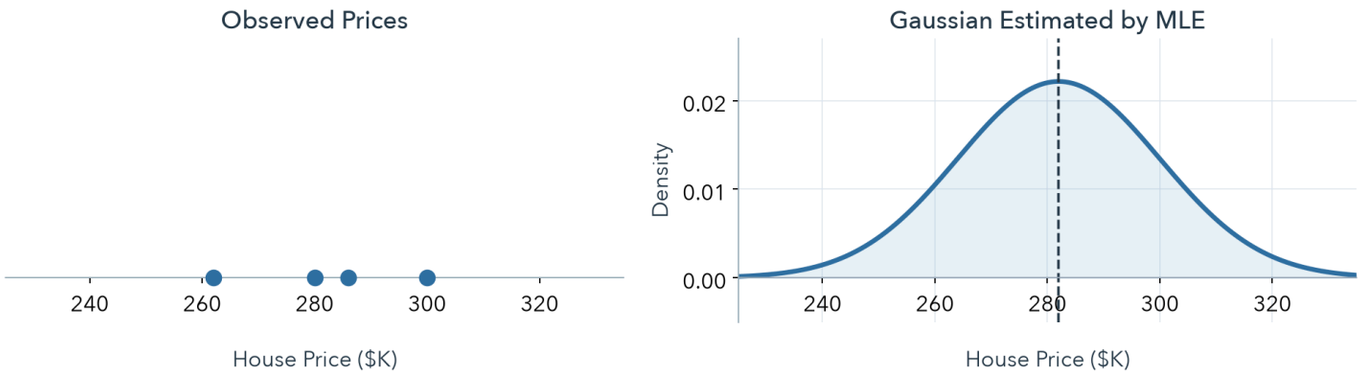
Example: Gaussian House Prices

Prices of $100\,\mathrm{m}^2$ houses, in **$K**: $\mathcal{D} = (262, 280, 286, 300)$.

Model: independent $Y_i \sim \mathcal{N}(\mu, 18^2)$, with unknown mean μ.

$$-\log p(\mathcal{D} \mid \mu) = C + \frac{1}{2 \cdot 18^2} \sum_{i=1}^{4} (y_i - \mu)^2 \qquad (C \text{ independent of } \mu),$$

$$\widehat{\mu}_{\mathrm{MLE}} = \text{sample mean} = \frac{262 + 280 + 286 + 300}{4} = 282.$$



Maximum A Posteriori Estimation

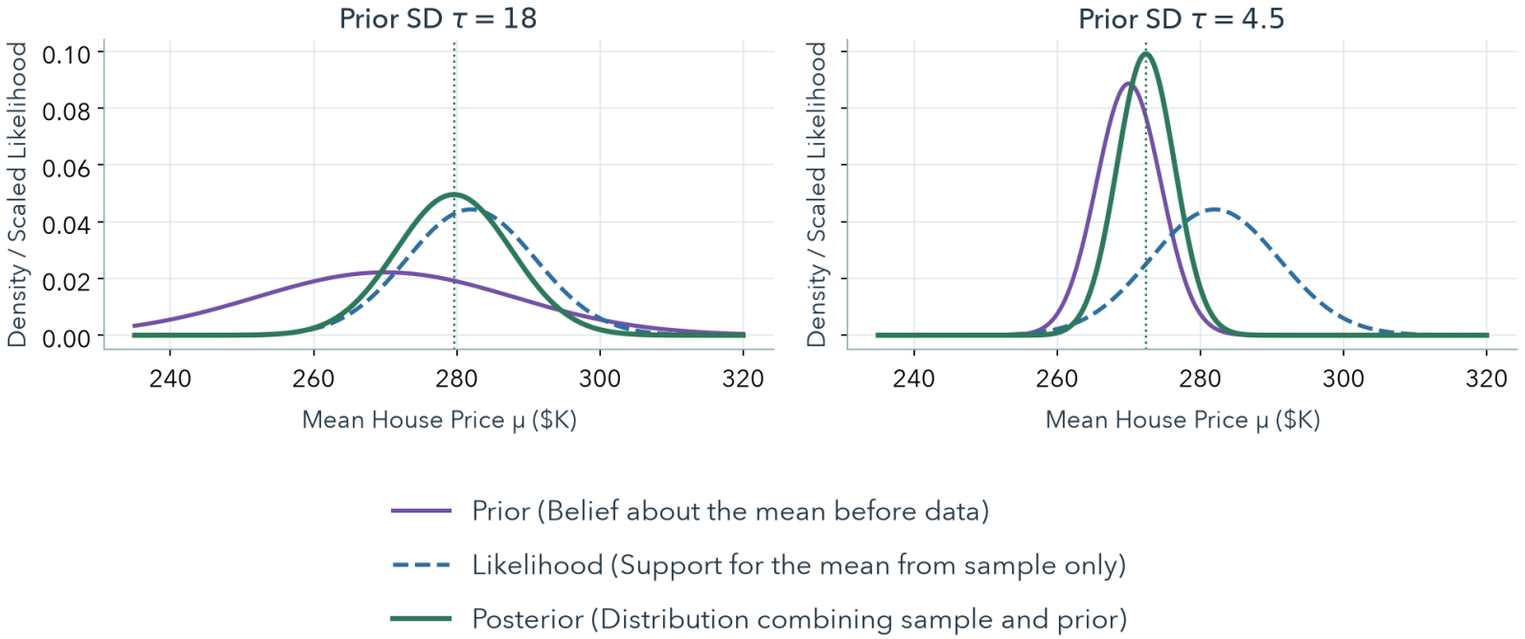
Maximum a posteriori (MAP) selects the parameter with the largest posterior density.

$$
\begin{aligned}
\hat{\theta}_{\mathrm{MAP}} &= \arg\max_{\theta} p(\theta \mid \mathcal{D}) \\
&= \arg\max_{\theta} \frac{p(\mathcal{D}|\theta)p(\theta)}{p(\mathcal{D})} \\
&= \arg\max_{\theta} p(\mathcal{D} \mid \theta)p(\theta).
\end{aligned}
$$

Example: House Prices with a Prior

The same four prices give $\widehat{\mu}_{\mathrm{MLE}} = 282$ (in **$K**).

Now suppose earlier sales suggest a mean around **$270K**: $\mu \sim \mathcal{N}(270, \tau^2)$.



MAP moves the estimate toward **$270K**. A narrower prior gives a stronger pull.

Recall: Gaussian MAP Gives Ridge Regression

For linear regression, use a Gaussian label model and independent Gaussian weight priors:

$$Y_i \mid \mathbf{x}_i, \mathbf{w} \sim \mathcal{N}(\mathbf{x}_i^\top \mathbf{w}, \sigma^2),$$
$$w_j \sim \mathcal{N}(0, \tau^2), \qquad j = 1, \ldots, d.$$

Recall: the MAP estimate minimizes the **ridge objective**.

$$\widehat{\mathbf{w}}_{\mathrm{MAP}} = \arg\min_{\mathbf{w}} \left[\frac{1}{n} \sum_{i=1}^{n} (y_i - \mathbf{x}_i^\top \mathbf{w})^2 + \frac{\lambda}{n} \sum_{j=1}^{d} w_j^2 \right], \qquad \lambda = \frac{\sigma^2}{\tau^2}.$$

Smaller prior variance τ^2 gives stronger shrinkage toward zero. The intercept w_0 is unpenalized.

Connecting MLE, MAP, and ERM

Estimation method	Observation model	Parameter prior	Result
MLE	Gaussian	–	Least squares
MAP	Gaussian	Gaussian slopes	Ridge
MAP	Gaussian	Laplace slopes	Lasso

Practice Q7: A prior for a coin probability

In a separate experiment, a coin has unknown head probability $0 < \theta < 1$. Its prior density is $p(\theta) = C\theta(1-\theta)^2$ for $0 < \theta < 1$, and zero otherwise. Here $C > 0$ is a constant that makes the density integrate to 1; you do not need to calculate it. Six independent flips produce four heads and two tails.

A. The posterior density is proportional to $\theta^5(1-\theta)^4$ for $0 < \theta < 1$.

B. The MAP estimate is $\theta = 6/11$.

C. The MAP estimate is $\theta = 5/9$.

D. The maximum-likelihood estimate is also $\theta = 5/9$.

Questions